Autonomous Line Following

This game involves using the camera to detect the specified color, and making the robot follow the pre-laid line.

1. Recognition Procedure

Subscribe to the camera node's topic to get the RGB image.

Find the line to be followed in the image and select it. Pick color to get its color threshold and extract its features. Calculate the robot's offset from the line's position in the field of view, and control the robot to move along the line. Keep the line in the center of the field of view by continuously correct the robot’s position. Use the lidar to detect the obstacle, enabling the obstacle avoidance.

2. Operation Steps

1) Start MentorPi and connect it to VNC.

2) Input the command and press Enter to disable the app auto-start service.

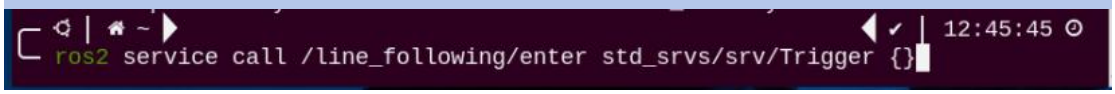
```
~/.stop_sh
```

3) Input the command to enable the line following.

```
ros2 launch app line_following_node.launch.py debug:=true
```

4) Open a new command line terminal. Run the following command to open the camera and select the target line.

```
ros2 service call /line_following/enter std_srvs/srv/Trigger {}
```



5) Open a new command line terminal again and enter the command. Press “Enter” to execute the line following.

```
ros2 service call /line_following/set_running std_srvs/srv/SetBool
```

```
"{data: True}"
```

```
ros2 service call /line_following/set_running std_srvs/srv/SetBool "{data: True}"
```

6) If you want to exit the game, press “Ctrl+C” in the terminal interface. After experiencing the game, you can enable the app service through commands or by restarting the robot. If the app is not enabled, the related app functions will not work. If the robot is restarted, the app will be automatically enabled.

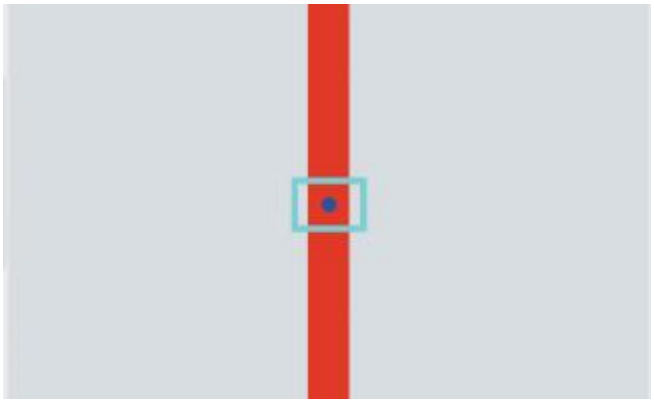
Click  and enter the command. Press enter to start the app, and wait for the buzzer to beep.

```
sudo systemctl restart start_node.service
```

```
sudo systemctl restart start_node.service
```

3. Program Outcome

After picking the color of the line to be followed, the robot starts to move along the line.



4. Program Analysis

The source code of the program is located in

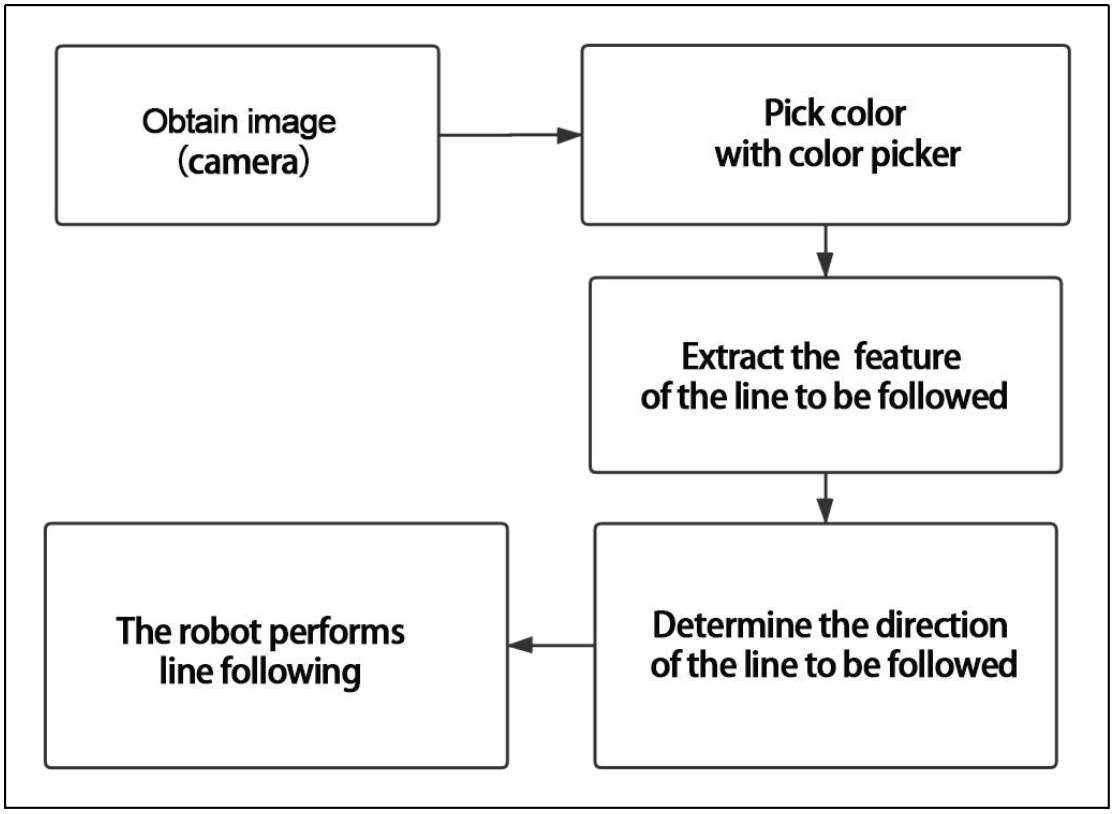
/home/ubuntu/ros2_ws/src/app/app/line_following.py

```python
    @staticmethod
    def get_area_max_contour(contours, threshold=100):
        '''
        获取最大面积对应的轮廓(get the contour of the largest area)
        :param contours:
        :param threshold:
        :return:
        '''
        contour_area = zip(contours, tuple(map(lambda c: math.fabs(cv2.contourArea(c)), contours)))
        contour_area = tuple(filter(lambda c_a: c_a[1] > threshold, contour_area))
        if len(contour_area) > 0:
            max_c_a = max(contour_area, key=lambda c_a: c_a[1])
            return max_c_a
        return None

    def __call__(self, image, result_image, threshold):
        centroid_sum = 0
        h, w = image.shape[:2]
        if os.environ['DEPTH_CAMERA_TYPE'] == 'ascamera':
            w = w + 200
        min_color = [int(self.target_lab[0] - 50 * threshold * 2),
                     int(self.target_lab[1] - 50 * threshold),
                     int(self.target_lab[2] - 50 * threshold)]
        max_color = [int(self.target_lab[0] + 50 * threshold * 2),
                     int(self.target_lab[1] + 50 * threshold),
                     int(self.target_lab[2] + 50 * threshold)]
        target_color = self.target_lab, min_color, max_color
        for roi in self.rois:
            blob = image[int(roi[0]*h):int(roi[1]*h), int(roi[2]*w):int(roi[3]*w)]  # 截取roi(intercept roi)
            img_lab = cv2.cvtColor(blob, cv2.COLOR_RGB2LAB)  # rgb转lab(convert rgb into lab)
            img_blur = cv2.GaussianBlur(img_lab, (3, 3), 3)  # 高斯模糊去噪(perform Gaussian filtering to reduce noise)
            mask = cv2.inRange(img_blur, tuple(target_color[1]), tuple(target_color[2]))  # 二值化(image binarization)
            eroded = cv2.erode(mask, cv2.getStructuringElement(cv2.MORPH_RECT, (3, 3)))  # 腐蚀(corrode)
            dilated = cv2.dilate(eroded, cv2.getStructuringElement(cv2.MORPH_RECT, (3, 3)))  # 膨胀(dilate)
            # cv2.imshow('section:{}:{}'.format(roi[0], roi[1]), cv2.cvtColor(dilated, cv2.COLOR_GRAY2BGR))
            contours = cv2.findContours(dilated, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_TC89_L1)[-2]  # 找轮廓(find the contour)
            max_contour_area = self.get_area_max_contour(contours, 30)  # 获取最大面积对应轮廓(get the contour corresponding to the largest contour)
            if max_contour_area is not None:
                rect = cv2.minAreaRect(max_contour_area[0])  # 最小外接矩形(minimum circumscribed rectangle)
                box = np.intp(cv2.boxPoints(rect))  # 四个角(four corners)
                for i in range(4):
```

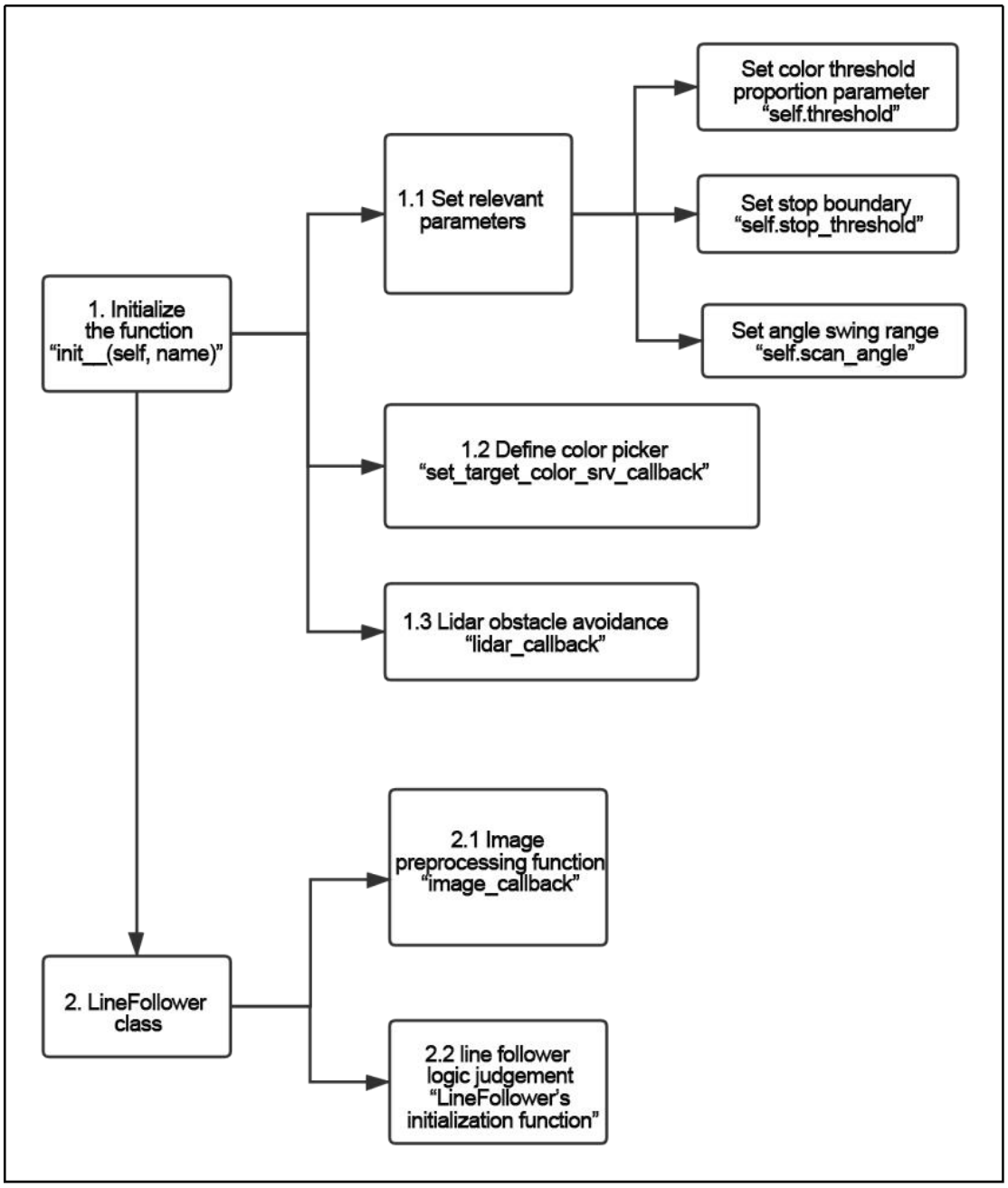
Note: it's necessary to back up the original program file before making any modification to the program. You're forbidden to modify the original source code file to avoid causing robot malfunctions that may be irreparable due to incorrect parameter modifications!

According to the game's effect, the process logic of this game is summarized as shown in the following diagram:



Subscribe to the topic messages published by the camera node to obtain

RGB images. From the image, identify and select the target line. Determine the color threshold by picking the color of the line. Based on the line's color information, extract the features of the line for line following. Calculate the robot's offset relative to the line's position in the field of view. Control the robot to move along the line segment, continuously correcting its position to keep the line at the center of the field of view and use lidar to detect obstacles and avoid them.



First, some preset parameters such as color threshold ratio, stop

threshold, and angle swing range are defined through the initialization function. These parameters can affect the final recognition effect. Then, the color picker object is defined and the additional lidar obstacle avoidance function is set. Next, the LineFollower class is implemented, which mainly includes the image preprocessing function and the logic judgment of line following.

- **Set related parameters of the initialization function "init__(self, name)"**

The "LineFollowingNode" class is used as the starting point of the program, and some basic parameters are defined in the initialization function. The color threshold ratio, stop threshold, and angle swing range need to be noted. All the relevant parameters and definitions are in the "LineFollowingNode" class, as shown in the figure below:

```
93 class LineFollowingNode(Node):
94     def __init__(self, name):
95         rclpy.init()
96         super().__init__(name, allow_undeclared_parameters=True, automatically_declare_para
97 
98         self.name = name
99         self.set_callback = False
100        self.is_running = False
101        self.color_picker = None
102        self.follower = None
103        self.scan_angle = math.radians(45)
104        self.pid = pid.PID(0.005, 0.001, 0.0)
105        self.empty = 0
106        self.count = 0
107        self.stop = False
108        self.threshold = 0.5
109        self.stop_threshold = 0.4
110        self.lock = threading.RLock()
111        self.image_sub = None
112        self.lidar_sub = None
113        self.image_height = None
114        self.image_width = None
115        self.bridge = CvBridge()
116        self.image_queue = queue.Queue(2)
117        #self.camera_type = os.environ['DEPTH_CAMERA_TYPE']
```

- **Set threshold ratio parameter "self.threshold":**

```
108         self.threshold = 0.5
```

The "self.threshold" parameter sets the weight ratio of color processing to binary image during the color picking. It is called through the initialization function of the "LineFollower" class, as shown in the figure below:

```
54          w = w + 200
55      min_color = [int(self.target_lab[0] - 50 * threshold * 2),
56                   int(self.target_lab[1] - 50 * threshold),
57                   int(self.target_lab[2] - 50 * threshold)]
58      max_color = [int(self.target_lab[0] + 50 * threshold * 2),
59                   int(self.target_lab[1] + 50 * threshold),
60                   int(self.target_lab[2] + 50 * threshold)]
61      target_color = self.target_lab, min_color, max_color
```

The above initialization function calls the “self.threshold” parameter, which sets the minimum and maximum color ranges of the “target_color” for subsequent binary processing. To achieve better recognition and processing results, the parameters can be kept at default.

- **Set stop threshold “self.stop_threshold”:**

```
self.threshold = 0.5
```

The “self.stop_threshold” judges when the robot should stop. As shown in the figure below:

```
284          if min_dist_left < self.stop_threshold or min_dist_right < self.stop_threshold:
285              self.stop = True
```

The above code determines when the car should stop. “min_dist_left” and “min_dist_right” calculate the distances from the robot to the left and right sides. When these two distances are less than the set stop threshold parameter, “self.stop” is set to True to stop the robot.

- **Set angle swing range “self.scan_angle”:**

```
103      self.scan_angle = math.radians(45)
```

The “self.scan_angle” is the swing angle of the robot during line following, with a range within 45 degrees. This angle parameter can achieve better line following effects while having a good scanning range. Increasing this parameter will increase the swing amplitude of the robot. Decreasing this parameter will decrease the swing amplitude of the robot. However, the recognition effect may not be optimal.

- **Define color picker “set_target_color_srv_callback”:**

The color picker is the tool for color picking in the app. It is defined using the “set_target_color_srv_callback” function, as shown in the figure below:

```
212     def set_target_color_srv_callback(self, request, response):
213         self.get_logger().info('\033[1;32m%s\033[0m' % "set_target_color")
214         with self.lock:
215             x, y = request.data.x, request.data.y
216             self.follower = None
217             if x == -1 and y == -1:
218                 self.color_picker = None
219             else:
220                 self.color_picker = ColorPicker(request.data, 5)
221                 self.mecanum_pub.publish(Twist())
222         response.success = True
223         response.message = "set_target_color"
224         return response
```

The “self.color_picker” is the definition of the picker. In the “ColorPicker” function, “req.data” is the position of the picked point. “20” is the length of the sent data frame. The function returns the feedback information of the set point.

- **Radar obstacle avoidance function “lidar_callback”:**

In the process of autonomous line following, there is also a lidar obstacle avoidance function. It is implemented in the “lidar_callback” function, as shown in the figure below:

```
256     def lidar_callback(self, lidar_data):
257         # 数据大小 = 扫描角度/每扫描一次增加的角度(data size= scanning angle/ the increased angle per scan)
258         if self.lidar_type != 'G4':
259             min_index = int(math.radians(MAX_SCAN_ANGLE / 2.0) / lidar_data.angle_increment)
260             max_index = int(math.radians(MAX_SCAN_ANGLE / 2.0) / lidar_data.angle_increment)
261             left_ranges = lidar_data.ranges[:max_index]  # 左半边数据(left data)
262             right_ranges = lidar_data.ranges[::-1][:max_index]  # 右半边数据(right data)
263         elif self.lidar_type == 'G4':
264             min_index = int(math.radians((360 - MAX_SCAN_ANGLE) / 2.0) / lidar_data.angle_increment)
265             max_index = int(math.radians(180) / lidar_data.angle_increment)
266             left_ranges = lidar_data.ranges[min_index:max_index][::-1]  # 左半边数据 (the left data)
267             right_ranges = lidar_data.ranges[::-1][min_index:max_index][::-1]  # 右半边数据 (the right data)
```

This is the setting of the radar scanning range. The scanning range should be set according to different lidar types. The parameters can be kept at default.

```
269         # 根据设定取数据(Get data according to settings)
270         angle = self.scan_angle / 2
271         angle_index = int(angle / lidar_data.angle_increment + 0.50)
272         left_range, right_range = np.array(left_ranges[:angle_index]), np.array(right_ranges[:angle_index])
273
```

The left and right point cloud data range processing: “angle” is the boundary line of the scanning range; “angle_index” is the point cloud data coordinate processing; “angle_increment” is the point cloud angle increment of the lidar scanning; “left_range” and “right_range” are the point cloud data of the left and right parts. The parameters can be kept at default.

```
278         # 取左右最近的距离(Take the nearest distance left and right)
279         min_dist_left_ = left_range[left_nonzero][left_nonan]
280         min_dist_right_ = right_range[right_nonzero][right_nonan]
281         if len(min_dist_left_) > 1 and len(min_dist_right_) > 1:
282             min_dist_left = min_dist_left_.min()
283             min_dist_right = min_dist_right_.min()
284             if min_dist_left < self.stop_threshold or min_dist_right < self.stop_threshold:
285                 self.stop = True
286             else:
287                 self.count += 1
288                 if self.count > 5:
289                     self.count = 0
290                     self.stop = False
```

This part judges the distance between the scanned point cloud data and obstacles. When the distance is less than the stop threshold, “self.stop” is set to True to stop the robot. Otherwise, the logic judgment is performed every 5 frames of data “self.cout > 5”.

- **“LineFollower” class:**

“LineFollower” is a class that contains the visual line following results and mainly focuses on the image preprocessing function and the logic judgment of line following.

- **“LineFollower” initialization function:**

In the program file, find the “LineFollower” class, whose initialization function includes the image preprocessing: color space conversion, Gaussian blur noise reduction, binarization, erosion and dilation, as shown in the figure below.

```
64          img_lab = cv2.cvtColor(blob, cv2.COLOR_RGB2LAB)  # rgb转lab(convert rgb into lab)
65          img_blur = cv2.GaussianBlur(img_lab, (3, 3), 3)  # 高斯模糊去噪(perform Gaussian filtering to reduce noise)
66          mask = cv2.inRange(img_blur, tuple(target_color[1]), tuple(target_color[2]))  # 二值化(image binarization)
67          eroded = cv2.erode(mask, cv2.getStructuringElement(cv2.MORPH_RECT, (3, 3)))  # 腐蚀(corrode)
68          dilated = cv2.dilate(eroded, cv2.getStructuringElement(cv2.MORPH_RECT, (3, 3)))  # 膨胀(dilate)
```

The “cvtColor” color space conversion converts the RGB image to the LAB image to reduce the interference of light. Gaussian blur noise reduction reduces the noise on the image.

“(3,3)” is the size of the noise convolution kernel, which can be kept at default. “cv2.inRange” is the binarization of the image, which presents the line following trajectory in black and white image. The erosion and dilation eliminate the spots of the selected image to obtain a clearer line following trajectory. If the recognition result is not ideal, the parameters of erosion and convolution can be appropriately increased. For specific parameter modification range, please refer to the relevant OpenCV lesson.

- **Change the style of the center point drawing:**

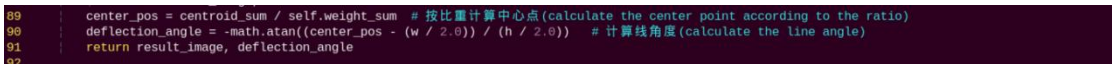
```
85              cv2.circle(result_image, (int(line_center_x), int(line_center_y)), 5, (0, 0, 255), -1)  # 画出中心点(draw the center point)
86              centroid_sum += line_center_x * roi[-1]
```

“line_center_x” and “line_center_y” are the coordinates of the calculated center point.

”cv2.circle” is the circle drawing function of OpenCV. The parameter “5” is the radius of the circle. Increasing this value will increase the radius of the circle. “(0, 0, 255)” represents the BGR ratio, representing the color of the drawn line, which is green. If it is changed to “(255, 0, 0)”, the color of the drawn line will be changed to red. “-1” represents a solid center point.

- **Line following logic judgment:**

The initialization function of “LineFollower” returns the line angle calculated, as shown in the figure below:



The “defection_angle” is the calculated line following angle. “math.atan” is the arctangent function; “center_pos” is the position of the specified line’s center point; “(w/2.0)/(h/2.0)” represents the center point of the image.